

Corrosion Rate Prediction ML Internship Study Guide

Industrial Asset Corrosion ML Pipeline

Dataset: 100,000 rows | Target: corrosion_rate_mpy

Models: Linear Regression | Random Forest | XGBoost

Tools: Python | scikit-learn | XGBoost | pandas | seaborn

Internship Presentation - 2026

1. Project Overview

This project builds a Machine Learning regression model to predict corrosion rate (mils per year, mpy) of industrial assets such as pipelines and tanks. Corrosion causes billions in losses. Early prediction enables planned maintenance, reduces failures, and cuts costs.

Business Problem

- Predict how fast an asset corrodes given its material, environment, and conditions.
- Help engineers prioritize which assets need urgent maintenance.
- Reduce unexpected failures and safety incidents.
- Optimize spending on coatings, inhibitors, and inspections.

Dataset Summary

- File: synthetic_corrosion_ml_dataset.xlsx | Sheet: in
- Rows: 100,000 | Columns: 33
- Split column: train=70,058 | val=14,931 | test=15,011
- Target: corrosion_rate_mpy (range: 0.04 to 56.77 mpy, mean=2.24)
- Missing: coating 14% and maintenance 55% - both filled as category "None"

Key Features

- Environmental: environment, temperature_c, ph, chloride_g_per_l, oxygen_ppm, humidity_pct
- Operational: flow_m_per_s, pressure_bar, exposure_days, inhibitor_ppm, base_aggressiveness
- Material: material, material_resistance_index
- Protection: coating, coating_efficiency, maintenance
- Geometry: wall_thickness_mm, diameter_mm, exposed_area_m2

2. Core Machine Learning Concepts

What is Machine Learning?

ML is where computers learn patterns from data to make predictions without explicit programming. Instead of hand-coded rules, you show thousands of examples and the algorithm learns the relationship between inputs (features) and output (target).

What is Regression?

Regression is supervised learning that predicts a continuous number. Classification predicts a category (yes/no, type A/B). Our target `corrosion_rate_mpy` is a continuous number, so we use Regression.

- Regression output: a NUMBER e.g. 2.5 mpy, house price 250000
- Classification output: a CATEGORY e.g. High Risk / Low Risk
- Formula: $y = f(x_1, x_2, \dots, x_n)$ where y is `corrosion_rate_mpy`

The ML Pipeline

- Step 1 - Data Collection: Gather raw data (our Excel file).
- Step 2 - EDA: Explore distributions, patterns, missing values.
- Step 3 - Preprocessing: Clean data, encode categories, scale features.
- Step 4 - Model Training: Fit algorithms to learn from training data.
- Step 5 - Evaluation: Measure prediction quality on unseen data.
- Step 6 - Insights: Extract feature importance and business recommendations.

EDA Key Findings

- Target is right-skewed: mean=2.24 mpy, median=1.30 mpy, max=56.77 mpy.
- Top correlations with target: `base_aggressiveness` (+0.47), `flow` (+0.45), `coating_efficiency` (-0.45).
- Carbon Steel corrodes at 3.96 mpy on average vs Ti Grade 2 at 0.85 mpy.
- CO2 Pipeline Wet is most aggressive: Carbon Steel hits 9.50 mpy in that environment.
- Uncoated assets corrode 3-4x faster than coated ones (coating "None" effect).

3. Data Preprocessing

Handling Missing Values

Missing values can cause errors or bias. Strategy depends on why data is missing:

- Mean/Median Imputation: Replace with average. Good for numeric, few missing.
- Mode Imputation: Replace with most frequent value. Good for categorical.
- New Category: If NaN means "not applied", treat it as a valid "None" category.
- Our approach: coating NaN -> "None" (no coating applied), maintenance NaN -> "None".

Categorical Encoding

ML algorithms work with numbers, not text. We convert categories to numbers:

- One-Hot Encoding (pd.get_dummies): Creates one binary column per category.
- Example: material=Carbon Steel -> material_Carbon Steel=1, all others=0.
- Label Encoding: Assigns integer 0,1,2... Risk: implies false ordering.
- Our approach: One-Hot for environment, material, coating, maintenance.

Feature Scaling

- StandardScaler (Z-score): Subtract mean, divide by std. Result: mean=0, std=1.
- MinMaxScaler: Scale to [0,1] range.
- Tree models (Random Forest, XGBoost) are scale-invariant - no scaling needed.
- Our approach: StandardScaler only for Linear Regression.

Train / Validation / Test Split

- Train (70%): Model learns parameters from this data.
- Validation (15%): Tune hyperparameters without touching test set.
- Test (15%): Final evaluation - model never sees this during training.
- Our dataset has a predefined "split" column - we use it for reproducibility.

4. Model 1 - Linear Regression

What is it?

Linear Regression assumes the target is a weighted sum of input features. It is the simplest regression model.

Formula: $y = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n + b$

y=prediction, w=learned weights, x=feature values, b=bias term.

How it learns

- Minimizes sum of squared differences between actual and predicted (OLS).
- Has a closed-form analytical solution - no iteration needed.
- Positive weight: feature increases corrosion rate.
- Negative weight: feature decreases corrosion rate.

Assumptions

- Linear relationship between features and target.
- No multicollinearity (features not highly correlated with each other).
- Residuals are normally distributed with constant variance (homoscedasticity).
- Requires feature scaling for fair weight comparison.

Pros and Cons

- PRO: Simple, fast, highly interpretable.
- PRO: Works well when the relationship is truly linear.
- CON: Cannot capture non-linear relationships.
- CON: Sensitive to outliers and feature scale.
- CON: Poor on complex datasets - corrosion has strong non-linear effects.
- Expected R2 on our data: ~0.45 (serves only as a baseline).

5. Model 2 - Random Forest Regressor

What is a Decision Tree?

A Decision Tree splits data based on feature conditions like a flowchart, until reaching leaf nodes that contain predictions. Example: IF temperature_c > 50 AND ph < 5 THEN predict corrosion_rate = HIGH.

What is Random Forest?

Random Forest is an ensemble of many Decision Trees, each trained on random subsets of data. The final prediction is the average of all trees.

- Bootstrap Sampling: Each tree trains on a random sample of rows (with replacement).
- Feature Randomness: At each split, only a random subset of features is considered.
- These two make trees diverse and collectively reduce overfitting.
- This technique is called Bagging (Bootstrap Aggregating).

Key Hyperparameters Tuned

- n_estimators: Number of trees (200-500). More trees = more accurate but slower.
- max_depth: Max depth of each tree. Limits complexity, prevents overfitting.
- min_samples_leaf: Min samples in a leaf. Higher = smoother, more general predictions.
- max_features: Features considered at each split. sqrt(n_features) is default.

Pros and Cons

- PRO: Handles non-linear relationships very well.
- PRO: Robust to outliers and noisy features.
- PRO: Built-in feature importance ranking.
- PRO: No scaling needed (trees are scale-invariant).
- CON: Slower to train (hundreds of trees).
- CON: Less interpretable than a single Decision Tree.
- Expected R2 on our data: ~0.88-0.90.

6. Model 3 - XGBoost (Extreme Gradient Boosting)

What is Boosting?

Boosting builds trees sequentially where each new tree corrects the errors of the previous trees. Unlike Random Forest (parallel), boosting is sequential.

How Gradient Boosting Works

- Start with a simple prediction (mean of target).
- Calculate residuals: actual - predicted.
- Train next tree to predict these residuals.
- Add new tree to ensemble with small learning rate (shrinkage).
- Repeat for $n_{\text{estimators}}$ rounds.

Why XGBoost is Powerful

- Regularization L1 (reg_alpha) and L2 (reg_lambda): Prevents overfitting.
- Tree pruning: Removes splits that do not improve the model.
- Native handling of missing values - learns best direction automatically.
- Parallel processing within each tree (column-level parallelism).
- Early stopping: Stop when validation performance stops improving.

Key Hyperparameters Tuned

- $n_{\text{estimators}}$: Number of boosting rounds (300-1000).
- learning_rate: Shrinkage per tree (0.01-0.1). Lower = needs more trees.
- max_depth: Tree depth (6-10). Controls complexity.
- subsample: Row fraction per tree (0.7-0.9). Adds randomness.
- colsample_bytree: Feature fraction per tree (0.7-0.9).
- reg_alpha / reg_lambda: L1 / L2 regularization strength.

XGBoost vs Random Forest

- XGBoost: Sequential, corrects errors iteratively. Usually higher accuracy.
- Random Forest: Parallel, averages independent trees. Faster to train.
- XGBoost more hyperparameter-sensitive but more powerful when tuned.
- Expected R^2 on our data (tuned XGBoost): ~0.93.

7. Model Evaluation Metrics

Metrics tell us how well the model predicts. Use multiple metrics for the full picture.

MAE - Mean Absolute Error

$MAE = \text{mean}(|\text{actual} - \text{predicted}|)$

- Intuitive: "On average predictions are off by X mpy".
- Robust to outliers (absolute, not squared).
- Same unit as target (mpy). Lower is better.
- Our XGBoost result: MAE ~ 0.33 mpy.

RMSE - Root Mean Squared Error

$RMSE = \sqrt{\text{mean}((\text{actual} - \text{predicted})^2)}$

- Penalizes large errors more heavily than MAE (due to squaring).
- More sensitive to outliers than MAE.
- Always $RMSE \geq MAE$. Large gap = outliers present.
- Our XGBoost result: RMSE ~ 0.77 mpy.

R2 - R-Squared (Coefficient of Determination)

$R^2 = 1 - (\text{sum of squared residuals}) / (\text{total variance})$

- $R^2 = 1.0$: Perfect predictions.
- $R^2 = 0.0$: Model no better than predicting the mean.
- $R^2 < 0$: Worse than predicting the mean.
- Our XGBoost result: $R^2 \sim 0.93$ (explains 93% of variance). Excellent!

MAPE - Mean Absolute Percentage Error

$MAPE = \text{mean}(|\text{actual} - \text{predicted}| / \text{actual}) \times 100\%$

- Expresses error as percentage of actual value.
- Easy to communicate to non-technical stakeholders.
- Unstable when actual values are near zero.

Metric Selection Guide

- Use R^2 to compare models quickly (always higher = better).
- Use MAE to report to business (in mpy - easy to understand).
- Use RMSE when large prediction errors are especially costly.
- Use MAPE for percentage-based reporting to management.

8. Hyperparameter Tuning

Parameters vs Hyperparameters

- Parameters: Values the MODEL learns from data (split thresholds, weights).
- Hyperparameters: Values YOU set before training (n_estimators, max_depth).
- Hyperparameters control the learning process - not learned from data.

RandomizedSearchCV

Randomly samples hyperparameter combinations and evaluates each with Cross-Validation. More efficient than GridSearchCV (tries all combinations).

- Cross-Validation: Splits train data into k folds, trains on k-1, validates on 1.
- Repeats k times rotating which fold is held out. Averages scores = robust estimate.
- n_iter=30: Try 30 random combinations. GridSearch on same space = 500+ combinations.
- Scoring: neg_mean_absolute_error (negate MAE so higher=better convention).

Tuning Decisions in Our Project

- XGBoost: n_estimators, max_depth, learning_rate, subsample, colsample_bytree, reg_alpha, reg_lambda.
- Random Forest: n_estimators, max_depth, min_samples_split, min_samples_leaf, max_features.
- CV folds: 3 (balances speed vs reliability on 100K rows).

9. Feature Importance & Business Insights

What is Feature Importance?

Feature importance tells us which inputs most influence predictions. In XGBoost it is measured by "Gain" - average improvement in loss from splits using that feature across all trees. Higher = more important.

Top 10 Features (Our Model)

- 1. base_aggressiveness: Overall aggressiveness score of the environment.
- 2. coating_efficiency: How well the protective coating works (0-1 scale).
- 3. material_resistance_index: Materials inherent resistance to corrosion.
- 4. temperature_c: Higher temp accelerates electrochemical reactions.
- 5. ph: Lower pH (more acidic) = faster corrosion.
- 6. flow_m_per_s: Higher flow removes protective oxide layer (erosion-corrosion).
- 7. chloride_g_per_l: Chlorides break down protective films on steel.
- 8. inhibitor_ppm: Chemical inhibitor concentration reduces corrosion rate.
- 9. exposure_days: Longer exposure = more cumulative corrosion.
- 10. humidity_pct: High humidity enables atmospheric electrochemical corrosion.

Business Recommendations

- MATERIAL: Use Duplex 2205 or Ti Grade 2 for high-risk assets (CO2, Brine zones).
- COATING: Regular inspection - 3LPE/FBE coatings reduce corrosion by 60-80%.
- CHEMISTRY: Control pH > 6 and minimize chlorides in process fluids.
- INHIBITORS: Optimize dosing schedule - cost-effective vs failure costs.
- MONITORING: Focus inspection effort on CO2 Pipeline Wet and Process Brine zones.
- MAINTENANCE: Even basic Periodic Wash significantly reduces corrosion rate.

10. Results Summary

Model Performance on Validation Set (14,931 rows)

- Linear Regression: $R^2 \sim 0.45$ | MAE ~ 1.20 mpy | RMSE ~ 2.10 mpy (Baseline)
- Random Forest: $R^2 \sim 0.88$ | MAE ~ 0.45 mpy | RMSE ~ 1.00 mpy (Good)
- XGBoost: $R^2 \sim 0.91$ | MAE ~ 0.38 mpy | RMSE ~ 0.88 mpy (Better)
- XGBoost Tuned: $R^2 \sim 0.93$ | MAE ~ 0.33 mpy | RMSE ~ 0.77 mpy (Best)

Why XGBoost Wins

- Corrosion is non-linear - temperature, pH, and material interact in complex ways.
- Boosting iteratively corrects errors, capturing non-linear interactions.
- Regularization prevents overfitting on the 100K-row dataset.
- Feature importance aligns with known electrochemical corrosion science.

Output Files Generated

- best_xgboost_model.pkl - Saved trained model for future predictions.
- scaler.pkl - Saved StandardScaler for consistent preprocessing.
- feature_names.pkl - Feature list in correct order for model input.
- test_predictions.csv - Actual vs predicted on test set with residuals.
- val_predictions.csv - Actual vs predicted on validation set.
- heatmap_material_vs_features.png / environment.png / coating.png - Visualizations.

11. Expected Professor Q&A

Q: What is the difference between regression and classification?

A: Regression predicts a continuous number (e.g. 2.5 mpy). Classification predicts a category (e.g. High/Low Risk). `corrosion_rate_mpy` is continuous, so we use regression.

Q: Why did you choose XGBoost as the final model?

A: XGBoost achieved the best R^2 (~0.93) and lowest MAE (~0.33 mpy) after tuning. Corrosion has non-linear feature interactions (pH effect changes with temperature) that XGBoost captures through sequential boosting. Linear Regression cannot do this.

Q: What is overfitting and how did you prevent it?

A: Overfitting is when a model memorizes training noise and fails on new data. Prevention: (1) `reg_alpha/reg_lambda` regularization in XGBoost, (2) `max_depth` limits tree complexity, (3) `subsample` adds randomness, (4) separate validation/test sets detect overfitting early.

Q: Why use the predefined split column instead of random splitting?

A: The predefined split ensures all models are compared on identical train/val/test sets. Random splits give different sets each run, making model comparison unreliable. Predefined splits also ensure reproducibility of all reported results.

Q: What is feature importance and why does it matter?

A: Feature importance measures each features contribution to reducing prediction error. In XGBoost it is the average gain from splits on that feature across all trees. It reveals which factors drive corrosion most, guiding maintenance priorities.

Q: How did you handle missing values in coating and maintenance?

A: Coating had 14% missing, maintenance 55% missing. We treated NaN as a new category "None" meaning no coating applied or no maintenance performed. This is semantically correct and preserves information - uncoated assets corrode much faster.

Q: What is Cross-Validation and why use it for tuning?

A: CV splits training data into k folds, training/validating k times rotating the held-out fold. This gives a robust performance estimate vs a single validation split. In `RandomizedSearchCV`, CV compares hyperparameter combinations fairly.

Q: What does $R^2=0.93$ mean in practice?

A: The model explains 93% of the variance in corrosion rate. Only 7% is unexplained (noise, unmeasured factors). For engineering applications, $R^2 > 0.90$ is excellent.

Q: What are the limitations of this model?

A: (1) Synthetic data - real corrosion has more noise and measurement error. (2) Cannot extrapolate beyond training data ranges. (3) Static features do not capture corrosion time dynamics (pit growth over time). (4) Feature interactions outside training distribution may not predict correctly.

Q: What is the difference between Random Forest and XGBoost?

A: Random Forest: Parallel training of many independent trees, average predictions (Bagging). XGBoost: Sequential training where each tree corrects previous errors (Boosting). XGBoost is usually more accurate but more sensitive to hyperparameters.

12. Quick Reference Cheat Sheet

Key Formulas

- MAE = $\text{mean}(|y_{\text{actual}} - y_{\text{pred}}|)$
- RMSE = $\sqrt{\text{mean}((y_{\text{actual}} - y_{\text{pred}})^2)}$
- $R^2 = 1 - \text{SS}_{\text{residuals}} / \text{SS}_{\text{total}}$
- MAPE = $\text{mean}(|y_{\text{actual}} - y_{\text{pred}}| / |y_{\text{actual}}|) \times 100\%$
- Linear Regression: $y = w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n + b$
- StandardScaler: $z = (x - \text{mean}) / \text{std}$

Model Comparison

- Linear Regression: No non-linear, needs scaling, fast, low accuracy, high interpretability.
- Random Forest: Handles non-linear, no scaling, medium speed, high accuracy.
- XGBoost: Handles non-linear, no scaling, medium speed, highest accuracy.

Key Python Libraries

- pandas: Data loading, manipulation, groupby, pivot_table.
- numpy: Numerical operations, array math.
- matplotlib / seaborn: Visualizations - heatmaps, scatter, histograms.
- scikit-learn: LinearRegression, RandomForestRegressor, StandardScaler, metrics, RandomizedSearchCV.
- xgboost: XGBRegressor - optimized gradient boosting.
- joblib: Save and load Python objects (models, scalers).

Core Concepts Glossary

- Supervised Learning: Train on labeled data (features + target).
- Regression: Predict a continuous number.
- Overfitting: Model memorizes training noise, fails on new data.
- Regularization: Penalty on complexity to prevent overfitting.
- Bagging (RF): Many independent trees, average predictions.
- Boosting (XGB): Sequential trees, each correcting the last.
- Feature Importance: How much each feature reduces prediction error.
- Cross-Validation: Rotate held-out fold for robust performance estimate.
- Hyperparameters: Settings chosen before training, not learned from data.
- One-Hot Encoding: Convert text categories to binary (0/1) columns.